Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №2

Децентралізовані застосунки

Робота з брокерами повідомлень

Виконав
аспірант групи ІТ-41ф

Новиков Д. М.

Перевірив:

д.т.н. Гаврилко Євген Володимирович

Київ 2026

Мета роботи: Використовуючи роботу з брокерами повідомлень, розробити розподілений веб-додаток відповідно до індивідуального завдання.

Завдання роботи:

Розподілений веб-додаток розбити на 2 модулі:

- перший модуль приймає запит від клієнта, наприклад REST або GRPC, валідує і відправляє через систему обміну повідомлення другої частини;

- другий модуль - забирає повідомлення з черги та зберігає у БД.

Система управління чергами, наприклад, RabbitMQ.

Для швидкого встановлення та налаштування Rabbit рекомендується застосувати образ Docker.

Для передачі даних використовувати JSON або XML згідно з варіантом індивідуальних завдань.

Варіант 4:

4. Лікарські препарати.

Лікарські препарати мають такі властивості.

• Name – найменування препарату.

• Pharm – фірма-виробник.

• Group – група препаратів, до яких належать ліки (антибіотики, болезаспокійливі, вітаміни тощо).

• Analogs (можливо кілька) – містить назву аналога.

• Versions – варіанти виконання (консистенція/вид: таблетки, капсули, порошок, краплі тощо). Для кожного варіанту виконання може бути кілька виробників лікарських засобів з такими характеристиками:

• Certificate – свідоцтво про реєстрацію препарату (номер, дати видачі/витікання дії, реєструюча організація);

• Package – упаковка (тип упаковки, кількість упаковки, ціна за упаковку);

• Dosage – дозування препарату, періодичність прийому.

Кореневий елемент називається Medicine.

Хід роботи:

Використані технології:

- Платформа: .NET 9 (C#);
- Брокер повідомлень: RabbitMQ (протокол AMQP 0-9-1) з використанням асинхронного клієнта RabbitMQ.Client v7+;
- База даних та ORM: PostgreSQL 17 та Entity Framework Core (Provider: Npgsql);
- Інфраструктура: Docker та Docker Compose.

Архітектура Системи:

Система побудована на базі подієво-орієнтованої архітектури (Event-Driven) і складається з чотирьох ключових елементів:

- API (Medicine Producer API): Веб-сервіс (ASP.NET Core), який приймає складні JSON-об'єкти через REST-інтерфейс, валідує їх та відправляє у чергу RabbitMQ;
- RabbitMQ Broker: Центральний вузол обміну повідомленнями. Забезпечує гарантовану доставку та декуплезацію між прийомом даних та їх обробкою;
- Worker (Medicine Worker Service): Фоновий процес (Consumer), який асинхронно зчитує повідомлення з черги, виконує мапінг даних та зберігає їх у базу даних;
- DB (PostgreSQL): база даних для зберігання інформації про лікарські препарати.

Інфраструктура та Розгортання (Docker):

Використання Docker Compose дозволяє розгорнути API/Worker/RabbitMQ/ PostgreSQL DB однією командою.

- Автоматичні міграції: Система реалізує паттерн "Migration on Startup". При старті контейнера Worker автоматично перевіряє стан БД та застосовує нові міграції;
- Healthchecks та Dependencies: У Docker Compose налаштовані перевірки працездатності (healthcheck) для PostgreSQL. API та Worker очікують повної готовності бази даних до прийому запитів, перш ніж ініціалізувати власні з'єднання;
- Network Isolation: Усі компоненти взаємодіють всередині закритої віртуальної мережі Docker, звертаючись один до одного за іменами сервісів (db, rabbitmq), що забезпечує безпеку та зручність конфігурації через Environment Variables.

Приклад використання:

Для розгортання повної інфраструктури використовується сценарій docker-compose, який автоматично піднімає чотири взаємопов'язані контейнери: брокер повідомлень (RabbitMQ), базу даних (PostgreSQL), API та Worker.

docker-compose:

```
services:
  # Message Broker
  rabbitmq:
    image: rabbitmq:4.0-management
```

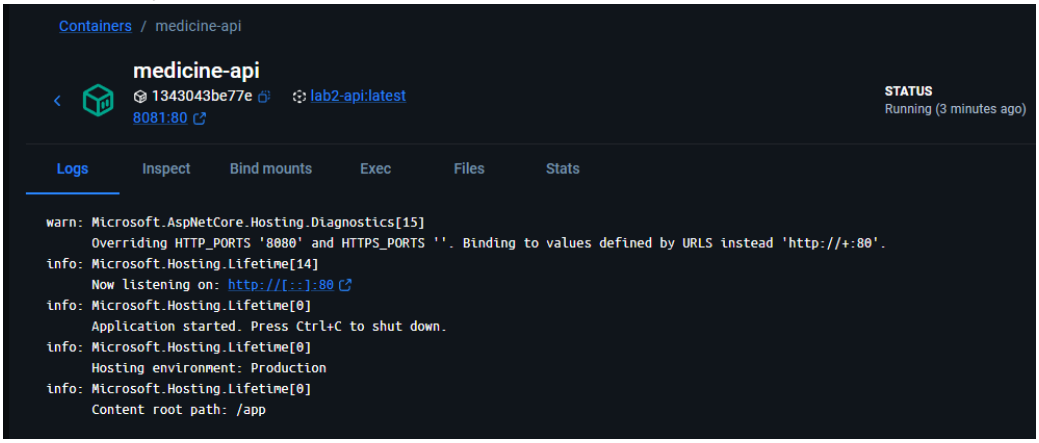
```yaml
    container_name: medicine-rabbitmq
    ports:
      - "5672:5672"
      - "15672:15672"
    environment:
      RABBITMQ_DEFAULT_USER: guest
      RABBITMQ_DEFAULT_PASS: guest

  # Database
  db:
    image: postgres:17
    container_name: medicine-db
    environment:
      POSTGRES_DB: MedicineDb
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d MedicineDb"]
      interval: 5s
      timeout: 5s
      retries: 5

  # Module 1: API
  api:
    build:
      context: .
      dockerfile: API/Dockerfile
    container_name: medicine-api
    ports:
      - "8081:80"
    environment:
      - ASPNETCORE_URLS=http://+:80
      - RabbitMq__HostName=rabbitmq
      - RabbitMq__UserName=guest
      - RabbitMq__Password=guest
      - RabbitMq__QueueName=medicine_queue
    depends_on:
      db:
        condition: service_healthy
      rabbitmq:
        condition: service_started

  # Module 2: Worker
  worker:
    build:
      context: .
      dockerfile: Worker/Dockerfile
    container_name: medicine-worker
    environment:
      -
ConnectionStrings__DefaultConnection=Host=db;Database=MedicineDb;Username=postgres
;Password=password
      - RabbitMq__HostName=rabbitmq
      - RabbitMq__UserName=guest
      - RabbitMq__Password=guest
      - RabbitMq__QueueName=medicine_queue
    depends_on:
```

```
    db:
      condition: service_healthy
    rabbitmq:
      condition: service_started
    api:
      condition: service_started
```

Перевіримо логи наших компонентів під час запуску:

- Worker: Воркер автоматично виявив відсутність таблиць, застосував Entity Framework міграції та створив схему (Medicines, Analogs, Makers). Після цього успішно підключився до черги medicine_queue:

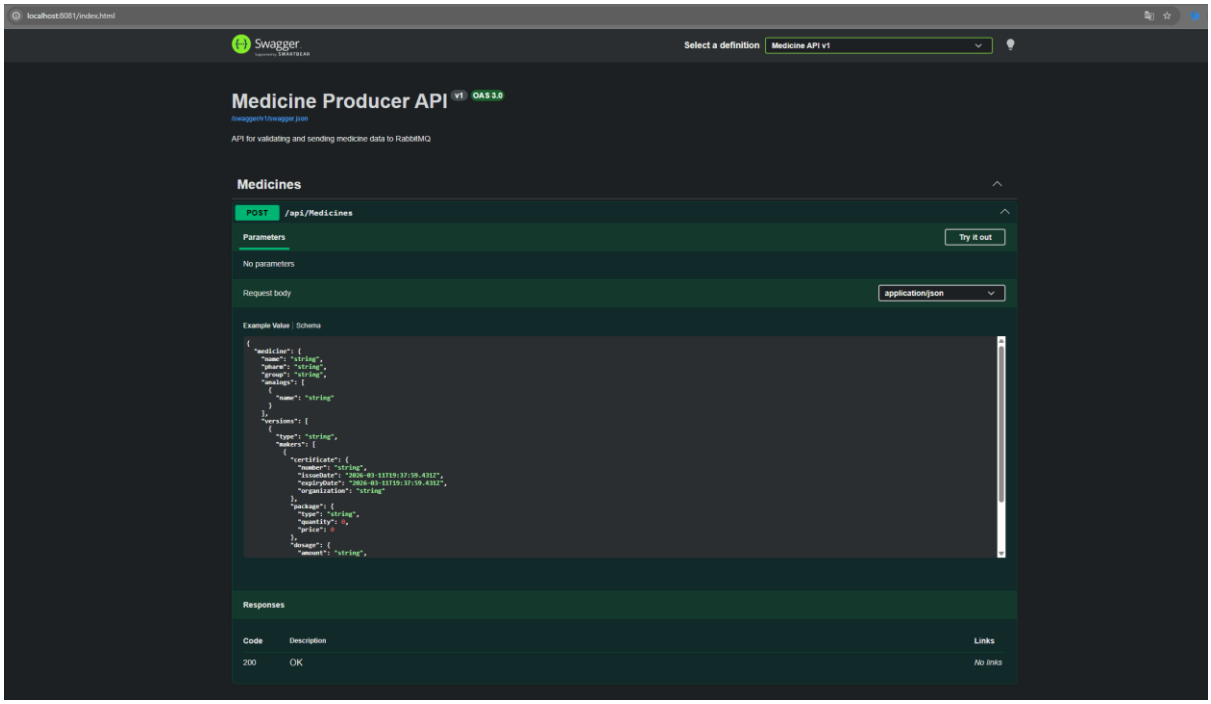
- API: Шлюз запустився на порту 8081. Веб-інтерфейс Swagger став доступним для ініціалізації запитів:



Сценарій взаємодії (Публікація та обробка даних):

Крок 1: Валідація повідомлення

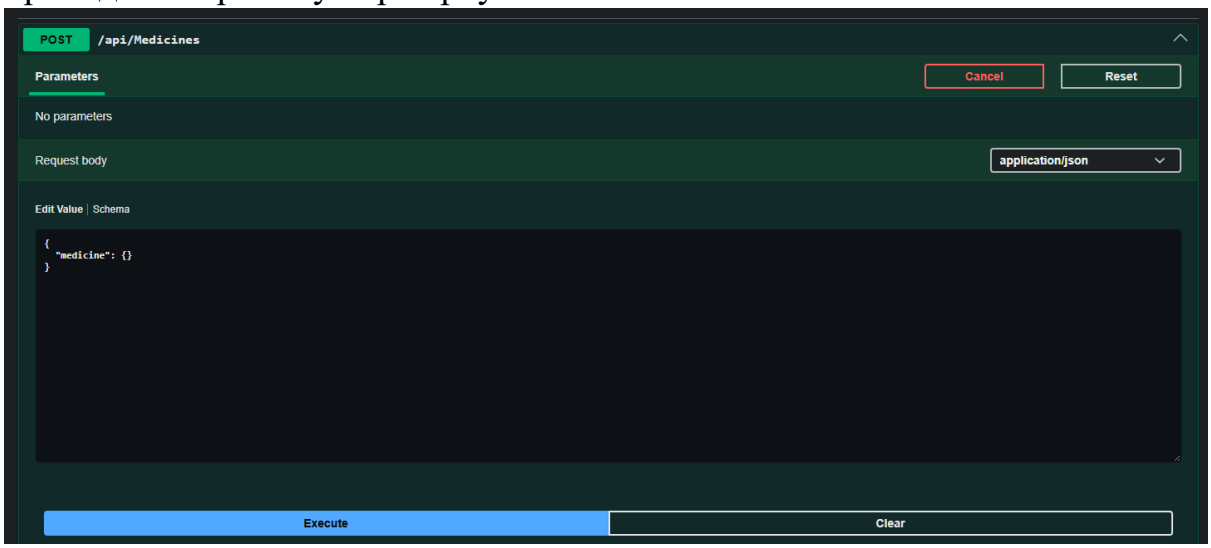
Відкриваємо інтерфейс Swagger (http://localhost:8081). Виконуємо POST-запит до ендпоінту /api/medicines. Система очікує JSON-об'єкт, що містить назву ліків, аналоги та інформацію про виробників (сертифікати, ціни, дозування):



Окрім перевірки успішної відправки, система реалізує механізм захисту від некоректних даних на двох рівнях: автоматичному (фреймворк) та логічному (код контролера).

Автоматична валідація (Data Annotations):

ASP.NET Core автоматично перевіряє відповідність вхідного JSON моделі DTO. Оскільки ми використовуємо атрибут [ApiController], фреймворк самостійно генерує відповідь 400 Bad Request, якщо запит не проходить первинну перевірку.



Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8081/api/Medicines' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "medicine": {}
}'
```

Request URL

http://localhost:8081/api/Medicines

Server response

Code	Details
400 *Undocumented*	Error: Bad Request

Response body


```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Medicine.Name": [
      "The Name field is required."
    ],
    "Medicine.Group": [
      "The Group field is required."
    ],
    "Medicine.Pharm": [
      "The Pharm field is required."
    ]
  },
  "traceId": "00-058ed85ff5aba4b2cd1de57b9a04f786-de88a67a14a1c9fd-00"
}
```


Download

Response headers

```
content-type: application/problem+json; charset=utf-8
date: Wed,11 Mar 2026 19:41:18 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	*No links*

На скріншоті видно результат спроби надіслати порожній об'єкт { "medicine": {} }. Фреймворк повертає об'єкт Problem Details де вказано, які саме обов'язкові поля відсутні. Це запобігає попаданню "сміттєвих" даних у чергу RabbitMQ.

Додаткова логічна перевірка в контролері:

Якщо запит структурно правильний, але містить логічні помилки, які важко описати атрибутами, у методі контролера реалізовано додаткову перевірку (перевірка на null та пусті рядки перед відправкою до брокера):

POST /api/Medicines

Parameters

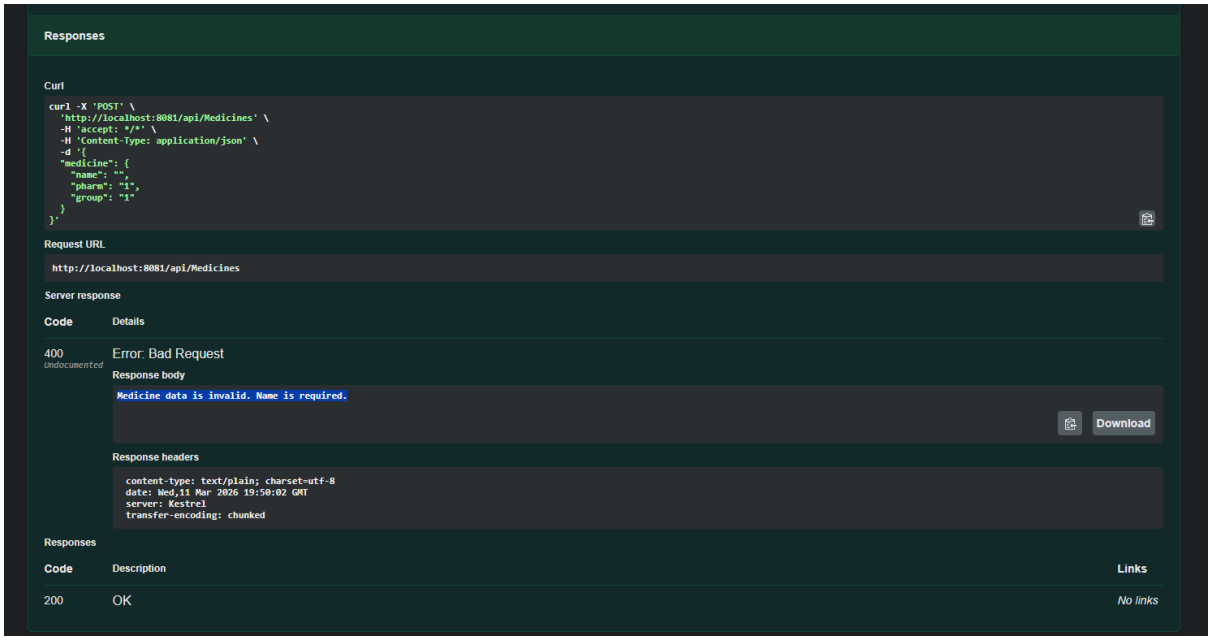
Cancel Reset

No parameters

Request body application/json

Edit Value | Schema

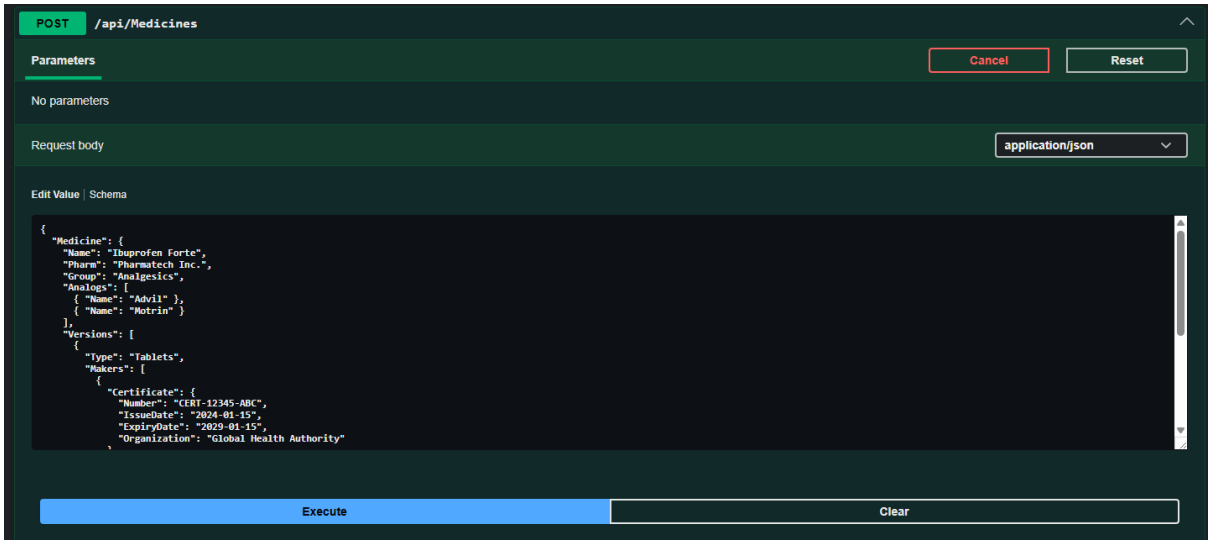
```
{
  "medicine": {
    "name": "",
    "pharm": "1",
    "group": "1"
  }
}
```

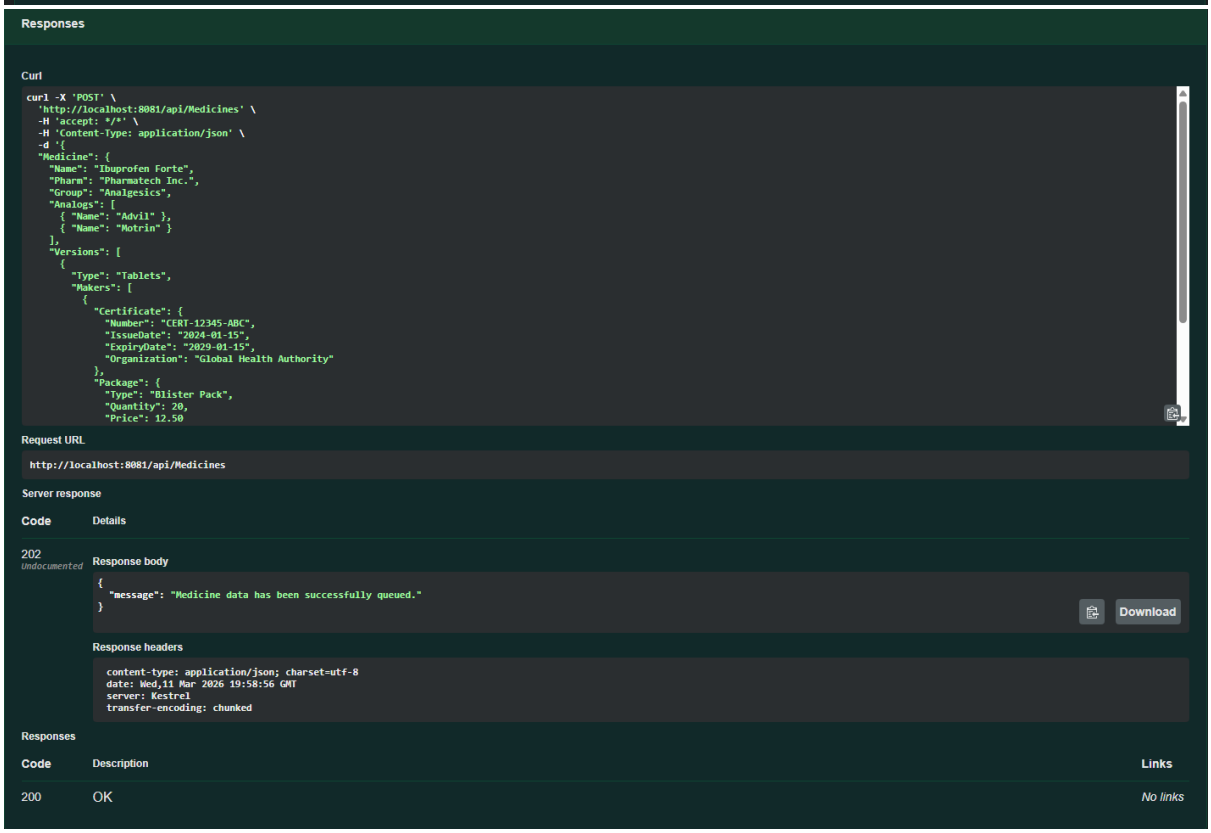


Крок 2: Відправка повідомлення (API -> RabbitMQ)

Цього разу надішлемо валідний JSON-об’єкт. Приклад вхідних даних:

```
{
  "Medicine": {
    "Name": "Ibuprofen Forte",
    "Pharm": "Pharmatech Inc.",
    "Group": "Analgesics",
    "Analogs": [
      { "Name": "Advil" },
      { "Name": "Motrin" }
    ],
    "Versions": [
      {
        "Type": "Tablets",
        "Makers": [
          {
            "Certificate": {
              "Number": "CERT-12345-ABC",
              "IssueDate": "2024-01-15",
              "ExpiryDate": "2029-01-15",
              "Organization": "Global Health Authority"
            },
            "Package": {
              "Type": "Blister Pack",
              "Quantity": 20,
              "Price": 12.50
            },
            "Dosage": {
              "Amount": "400mg",
              "Frequency": "1 tablet every 6-8 hours"
            }
          }
        ]
      }
    ]
  }
}
```



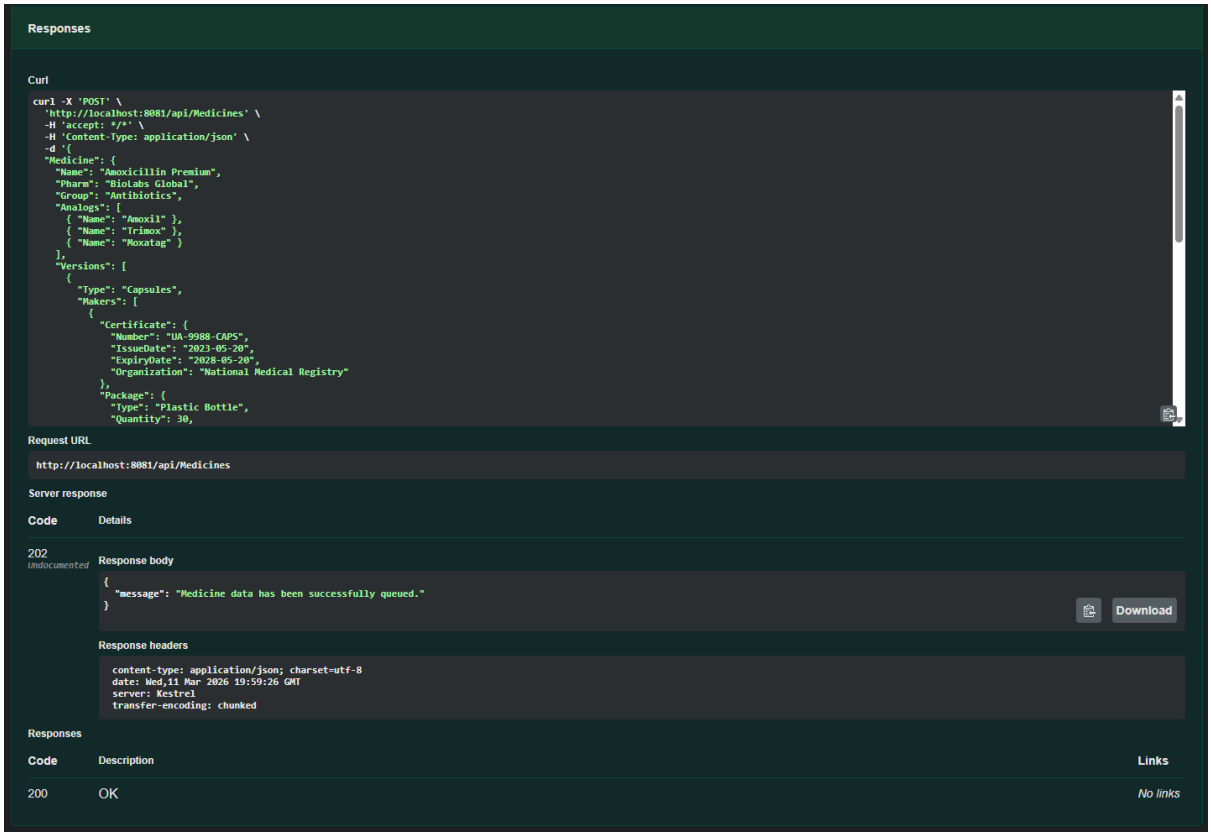


Після натискання Execute API перевіряє структуру, серіалізує об'єкт у JSON-рядок та публікує його в RabbitMQ. У відповідь отримуємо статус 200 OK.

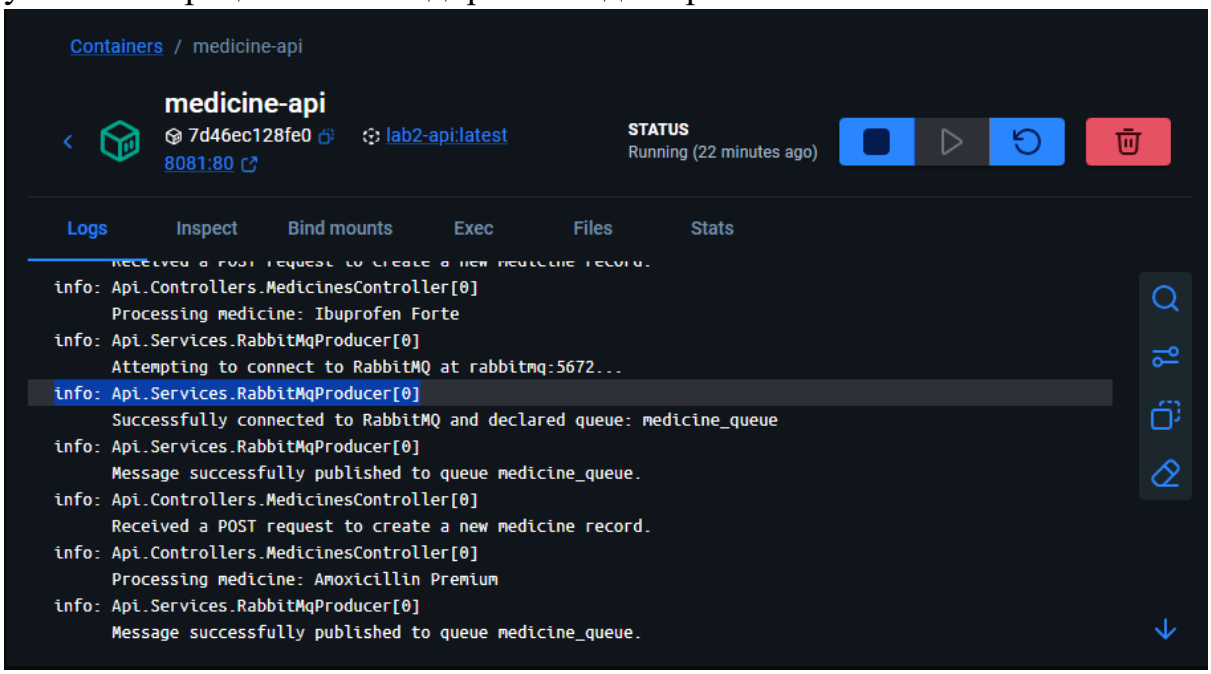
Для подальшого тестування додаємо ще один об’єкт (Amoxicillin Premium з декількома версіями випуску та аналогами):

```
{
  "Medicine": {
    "Name": "Amoxicillin Premium",
    "Pharm": "BioLabs Global",
    "Group": "Antibiotics",
    "Analogs": [
      { "Name": "Amoxil" },
      { "Name": "Trimox" },
      { "Name": "Moxatag" }
```

```
    ],
    "Versions": [
      {
        "Type": "Capsules",
        "Makers": [
          {
            "Certificate": {
              "Number": "UA-9988-CAPS",
              "IssueDate": "2023-05-20",
              "ExpiryDate": "2028-05-20",
              "Organization": "National Medical Registry"
            },
            "Package": {
              "Type": "Plastic Bottle",
              "Quantity": 30,
              "Price": 45.99
            },
            "Dosage": {
              "Amount": "500mg",
              "Frequency": "3 times a day for 7 days"
            }
          }
        ]
      },
      {
        "Type": "Oral Suspension",
        "Makers": [
          {
            "Certificate": {
              "Number": "UA-1122-LIQUID",
              "IssueDate": "2025-02-10",
              "ExpiryDate": "2030-02-10",
              "Organization": "National Medical Registry"
            },
            "Package": {
              "Type": "Glass Bottle",
              "Quantity": 1,
              "Price": 38.00
            },
            "Dosage": {
              "Amount": "250mg/5ml",
              "Frequency": "5ml every 12 hours"
            }
          }
        ]
      }
    ]
  }
}
```



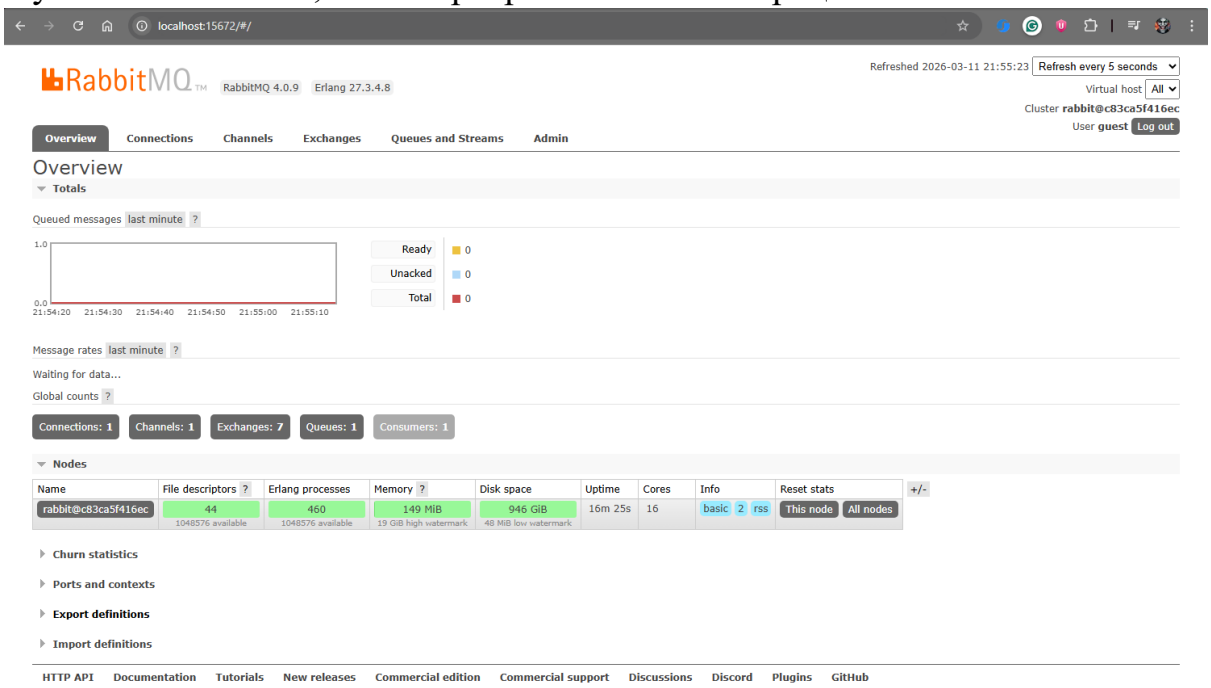
Перевіримо логи API, щоб переконатися, що повідомлення було успішно опрацьовано та відправлено до черги:

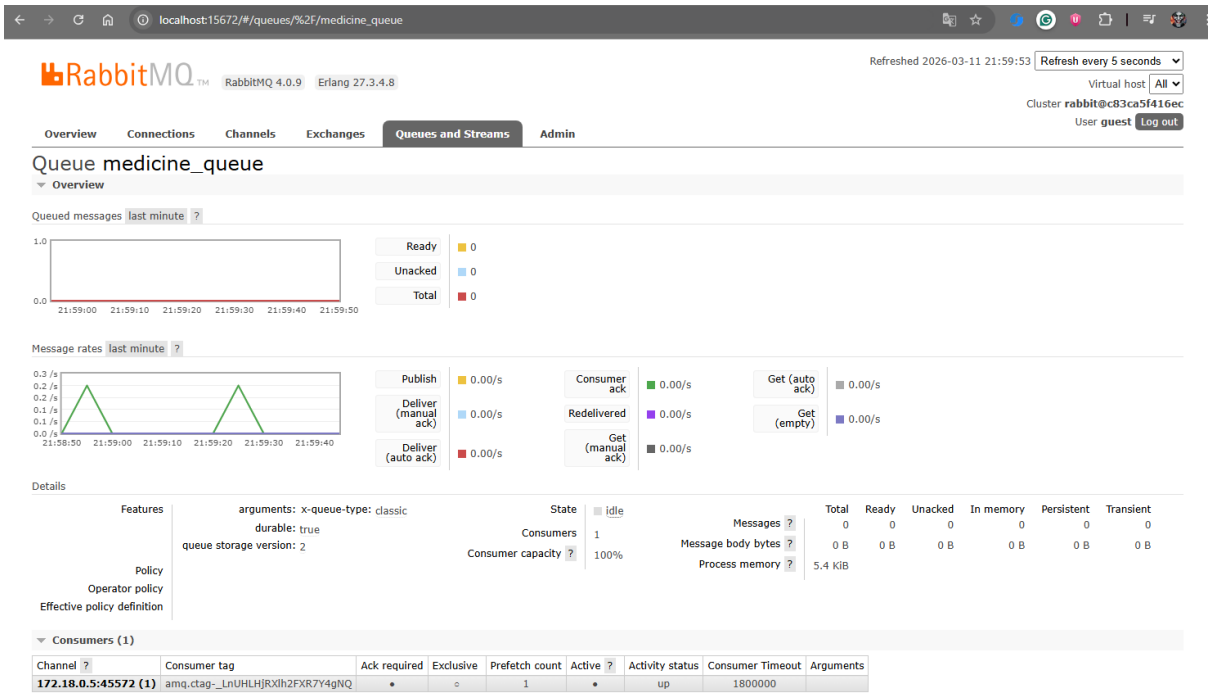


Крок 3: Моніторинг черги (RabbitMQ)

Переходимо в панель керування RabbitMQ (http://localhost:15672/). У розділі Queues бачимо активність у черзі medicine_queue. Оскільки

налаштувано ручне підтвердження (Manual Ack), повідомлення залишалося б у стані "Unacked", якби воркер не зміг його опрацювати:





Message rates (Графік інтенсивності): На графіку видно два зелені "піки". Вони відповідають двом окремим публікаціям повідомлень (Publish) з інтервалом приблизно у 35 секунд. Це підтверджує, що API успішно серіалізував об'єкти та передав їх брокеру.

Стан черги (Ready / Unacked): Показники Ready: 0 та Unacked: 0 свідчать про те, що повідомлення не затрималися в черзі. Оскільки Воркер

був активним у момент відправки, він миттєво "забрав" повідомлення на обробку та надіслав підтвердження (Ack), після чого RabbitMQ видалив їх із пам'яті.

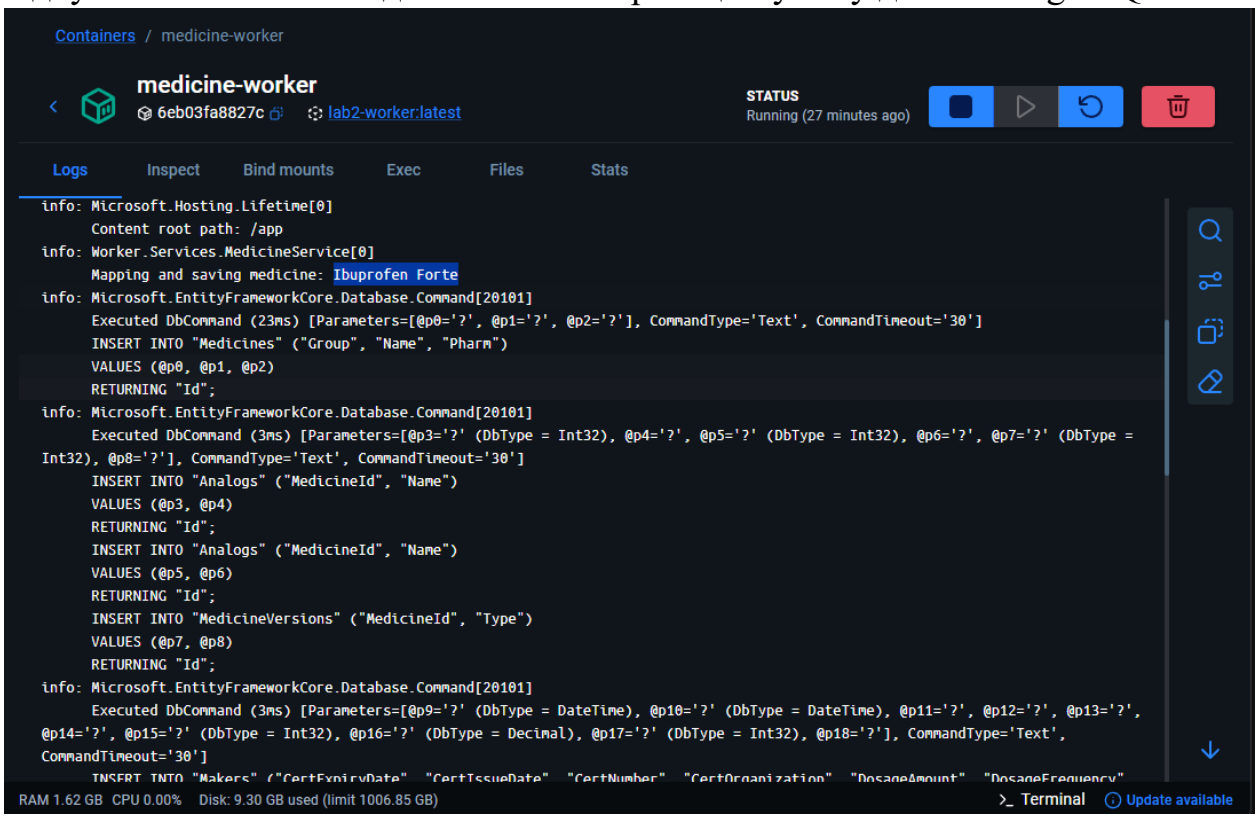
В нижній частині скріншоту відображена детальна інформація про підключений Worker Service:

- Consumers: 1 - підтверджує, що наш фоновий сервіс успішно ініціалізував з'єднання та готовий до роботи;
- Prefetch count: 1 - демонструє роботу механізму Quality of Service (QoS). Воркер бере в роботу лише одне повідомлення за раз, що запобігає перевантаженню пам'яті;
- Activity status: Up - канал зв'язку активний.

Дашборд RabbitMQ підтверджує стабільну роботу транспортного рівня системи. Відсутність повідомлень у стані "Ready" при наявності піків на графіку є прямим доказом того, що Worker успішно справляється з тестовим навантаженням

Крок 4: Асинхронна обробка та запис у БД (Worker)

На цьому етапі ми аналізуємо логи Worker, щоб зрозуміти, як саме відбувається запис складного об'єкта в реляційну базу даних PostgreSQL:



```
Containers / medicine-worker
medicine-worker
6eb03fa8827c  lab2-worker:latest
STATUS
Running (27 minutes ago)
Logs  Inspect  Bind mounts  Exec  Files  Stats
      INSERT INTO "Makers" ("CertExpiryDate", "CertIssueDate", "CertNumber", "CertOrganization", "DosageAmount", "DosageFrequency",
"MedicineVersionId", "PackagePrice", "PackageQuantity", "PackageType")
      VALUES (@p9, @p10, @p11, @p12, @p13, @p14, @p15, @p16, @p17, @p18)
      RETURNING "Id";
info: Worker.Services.MedicineService[0]
      Medicine Ibuprofen Forte successfully saved to DB.
info: Worker.Services.RabbitMqConsumer[0]
      Message Acked successfully.
```

```
Containers / medicine-worker
medicine-worker
6eb03fa8827c  lab2-worker:latest
STATUS
Running (27 minutes ago)
Logs  Inspect  Bind mounts  Exec  Files  Stats
      Medicine Ibuprofen Forte successfully saved to DB.
info: Worker.Services.RabbitMqConsumer[0]
      Message Acked successfully.
info: Worker.Services.MedicineService[0]
      Mapping and saving medicine: Amoxicillin Premium
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[@p0='?', @p1='?', @p2='?'], CommandType='Text', CommandTimeout='30']
      INSERT INTO "Medicines" ("Group", "Name", "Pharm")
      VALUES (@p0, @p1, @p2)
      RETURNING "Id";
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[@p3='?' (DbType = Int32), @p4='?', @p5='?' (DbType = Int32), @p6='?', @p7='?' (DbType =
Int32), @p8='?', @p9='?' (DbType = Int32), @p10='?', @p11='?' (DbType = Int32), @p12='?'], CommandType='Text', CommandTimeout='30']
      INSERT INTO "Analogs" ("MedicineId", "Name")
      VALUES (@p3, @p4)
      RETURNING "Id";
```

```
Containers / medicine-worker
medicine-worker
6eb03fa8827c  lab2-worker:latest
STATUS
Running (27 minutes ago)
Logs  Inspect  Bind mounts  Exec  Files  Stats
      VALUES (@p7, @p8)
      RETURNING "Id";
      INSERT INTO "MedicineVersions" ("MedicineId", "Type")
      VALUES (@p9, @p10)
      RETURNING "Id";
      INSERT INTO "MedicineVersions" ("MedicineId", "Type")
      VALUES (@p11, @p12)
      RETURNING "Id";
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[@p13='?' (DbType = DateTime), @p14='?' (DbType = DateTime), @p15='?', @p16='?', @p17='?',
@p18='?', @p19='?' (DbType = Int32), @p20='?' (DbType = Decimal), @p21='?' (DbType = Int32), @p22='?', @p23='?' (DbType = DateTime),
@p24='?' (DbType = DateTime), @p25='?', @p26='?', @p27='?', @p28='?', @p29='?' (DbType = Int32), @p30='?' (DbType = Decimal), @p31='?'
(DbType = Int32), @p32='?'], CommandType='Text', CommandTimeout='30']
      INSERT INTO "Makers" ("CertExpiryDate", "CertIssueDate", "CertNumber", "CertOrganization", "DosageAmount", "DosageFrequency",
"MedicineVersionId", "PackagePrice", "PackageQuantity", "PackageType")
      VALUES (@p13, @p14, @p15, @p16, @p17, @p18, @p19, @p20, @p21, @p22)
      RETURNING "Id";
      INSERT INTO "Makers" ("CertExpiryDate", "CertIssueDate", "CertNumber", "CertOrganization", "DosageAmount", "DosageFrequency",
"MedicineVersionId", "PackagePrice", "PackageQuantity", "PackageType")
      VALUES (@p23, @p24, @p25, @p26, @p27, @p28, @p29, @p30, @p31, @p32)
      RETURNING "Id";
info: Worker.Services.MedicineService[0]
      Medicine Amoxicillin Premium successfully saved to DB.
info: Worker.Services.RabbitMqConsumer[0]
      Message Acked successfully.
RAM 1.62 GB  CPU 0.62%  Disk: 9.30 GB used (limit 1006.85 GB)
>_ Terminal  Update available
```

Логи підтверджують правильну роботу Worker'у. Отримавши JSON, Entity Framework Core виконує batching запитів. Спочатку вставляється запис у головну таблицю "Medicines" з використанням RETURNING "Id".

Отримавши згенерований ідентифікатор бази даних, система миттєво використовує його як Foreign Key для наступних команд INSERT у таблиці "Analogs" та "MedicineVersions".

Всі ці SQL-команди виконуються в межах однієї атомарної транзакції. Система успішно перетворює деревоподібний JSON у плоскі реляційні таблиці, дотримуючись послідовності вставки згідно з ієрархією. Підтвердження повідомлення (Message Acked successfully) відбувається лише після успішного коміту транзакції в БД.

Крок 5: Фінальна верифікація даних (PostgreSQL)

Використовуємо інструмент Query Tool у pgAdmin для виконання серії запитів, щоб перевірити цілісність збережених даних, правильність роботи зовнішніх ключів та конвертацію типів:

1. Перевірка цілісності головної ієрархії (Medicine -> Versions -> Makers):

Цей запит об'єднує три таблиці, щоб підтвердити, що конкретна версія ліків правильно зв'язана з виробником та ціною.

Query Query History AI Assistant

```
SELECT
    m."Name" AS Medicine,
    v."Type" AS Version,
    mk."CertNumber",
    mk."PackagePrice"
FROM "Medicines" m
JOIN "MedicineVersions" v ON m."Id" = v."MedicineId"
JOIN "Makers" mk ON v."Id" = mk."MedicineVersionId";
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page

	medicine text	version text	CertNumber text	PackagePrice numeric
1	Ibuprofen Forte	Tablets	CERT-12345-ABC	12.50
2	Amoxicillin Premium	Capsules	UA-9988-CAPS	45.99
3	Amoxicillin Premium	Oral Suspension	UA-1122-LIQUID	38.00

2. Перевірка збереження списку аналогів (One-to-Many):

Запит підтверджує, що масив аналогів з JSON успішно розкладено на окремі рядки та прив'язано до правильного ідентифікатора препарату.

Query Query History AI Assistant

```sql
SELECT
    m."Name" AS Original_Medicine,
    a."Name" AS Analog_Name
FROM "Medicines" m
JOIN "Analogs" a ON m."Id" = a."MedicineId"
ORDER BY m."Name";
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page

	original_medicine text	analog_name text
1	Amoxicillin Premium	Amoxil
2	Amoxicillin Premium	Trimox
3	Amoxicillin Premium	Moxatag
4	Ibuprofen Forte	Advil
5	Ibuprofen Forte	Motrin

3. Глобальна вибірка для перевірки ієрархії:

Щоб побачити повну картину того, як вхідний JSON розклався на реляційну структуру, виконаємо комплексний запит, який об'єднує всі 4 таблиці бази даних в єдине представлення.

Query Query History AI Assistant Scratch Pad

```sql
WITH AnalogList AS (
    -- Групуємо всі аналоги для кожного препарату в один рядок
    SELECT "MedicineId", STRING_AGG("Name", ', ') AS "All_Analogs"
    FROM "Analogs"
    GROUP BY "MedicineId"
)
SELECT
    m."Id" AS "Medicine_ID",
    m."Name" AS "Medicine_Name",
    m."Pharm" AS "Manufacturer",
    m."Group" AS "Medical_Group",
    al."All_Analogs" AS "Analogs",
    v."Type" AS "Version_Type",
    mk."PackageType" AS "Package",
    mk."PackageQuantity" AS "Qty",
    mk."PackagePrice" AS "Price",
    mk."DosageAmount" AS "Dosage",
    mk."DosageFrequency" AS "Frequency",
    mk."CertNumber" AS "Certificate",
    mk."CertIssueDate" AS "Issued_At",
    mk."CertExpiryDate" AS "Expires_At",
    mk."CertOrganization" AS "Cert_Authority"
FROM "Medicines" m
LEFT JOIN AnalogList al ON m."Id" = al."MedicineId"
LEFT JOIN "MedicineVersions" v ON m."Id" = v."MedicineId"
LEFT JOIN "Makers" mk ON v."Id" = mk."MedicineVersionId"
ORDER BY m."Name", v."Type";
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	Medicin integer	Medicine_Name text	Manufacturer text	Medical_Group text	Analogs text	Version_Type text	Package text	Qty integer	Price numeric	Dosage text	Frequency text	Certificate text	Issued_At timestamp with time zone	Expires_At timestamp with time zone	Cert_Authority text
1	2	Amoxicillin Premium	BioLabs Global	Antibiotics	Amoxil, Trimox, Moxatag	Capsules	Plastic Bottle	30	45.99	500mg	3 times a day for 7 days	UA-9988-CAPS	2023-05-20 00:00:00+00	2028-05-20 00:00:00+00	National Medical Regis...
2	2	Amoxicillin Premium	BioLabs Global	Antibiotics	Amoxil, Trimox, Moxatag	Oral Suspension	Glass Bottle	1	38.00	250mg/5...	5ml every 12 hours	UA-1122-LIQUID	2025-02-10 00:00:00+00	2030-02-10 00:00:00+00	National Medical Regis...
3	1	Ibuprofen Forte	Pharmatech Inc.	Analgesics	Advil, Motrin	Tablets	Blister Pack	20	12.50	400mg	1 tablet every 6-8 hours	CERT-12345-A...	2024-01-15 00:00:00+00	2029-01-15 00:00:00+00	Global Health Authority

Цей сценарій підтверджує успішну реалізацію мікросервісної, подієво-орієнтованої архітектури (Event-Driven Architecture). Система успішно розділяє обов'язки між незалежними компонентами: API виконує функцію шлюзу для валідації та прийому даних, RabbitMQ виступає надійним буфером, а фоновий Worker бере на себе роботу зі зв'язування та транзакційного збереження реляційних ієрархій у PostgreSQL.

Висновки: за результатом виконання лабораторної роботи спроєктовано та реалізовано розподілену подієво-орієнтовану (Event-Driven) мікросервісну систему на базі брокера повідомлень RabbitMQ. Головною метою було створення надійного, слабкозв'язаного середовища для асинхронного прийому, валідації та збереження складних ієрархічних структур даних. Цю мету було досягнуто завдяки інтеграції REST API шлюзу (Producer) та фонового сервісу-обробника (Worker\Consumer). На стороні сервісу Worker реалізовано взаємодію з базою даних PostgreSQL за допомогою Entity Framework Core, налаштовано автоматичне застосування міграцій при старті та забезпечено повну оркестрацію всіх незалежних компонентів у віртуальній мережі контейнеризованого середовища Docker Compose.

Вихідний код застосунку можна знайти за наступним посиланням на GitHub.